```
try:
    import spikingjelly
except ImportError:
    !pip install spikingjelly
    print("spikingjelly installed.")

import torch
import torch.nn as nn
from spikingjelly.clock_driven.neuron import MultiStepLIFNode
from timm.models.layers import to_2tuple, trunc_normal_, DropPath
from timm.models.registry import register_model
from timm.models.vision_transformer import _cfg
import torch.nn.functional as F
from functools import partial

__all__ = ['Spikingformer']

class MLP(nn.Module):
    def __init__(self, in_features, hidden_features=None, out_features=None,
drop=0.):
        super().__init__()
        out_features = out_features or in_features
        hidden_features = hidden_features or in_features
        self.mlp1_lif = MultiStepLIFNode(tau=2.0, detach_reset=True,
backend='torch')
        self.mlp1_conv = nn.Conv2d(in_features, hidden_features, kernel_size=1,
stride=1)
        self.mlp1_bn = nn.BatchNorm2d(hidden_features)

        self.mlp2_lif = MultiStepLIFNode(tau=2.0, detach_reset=True,
backend='torch')
        self.mlp2_conv = nn.Conv2d(hidden_features, out_features, kernel_size=1,
stride=1)
        self.mlp2_bn = nn.BatchNorm2d(out_features)

        self.c_hidden = hidden_features
        self.c_output = out_features

    def forward(self, x):
        T,B,C,H,W = x.shape

        x = self.mlp1_lif(x)
        x = self.mlp1_conv(x.flatten(0,1))
        x = self.mlp1_bn(x).reshape(T,B,self.c_hidden,H,W).contiguous()

        x = self.mlp2_lif(x)
        x = self.mlp2_conv(x.flatten(0,1))
        x = self.mlp2_bn(x).reshape(T,B,C,H,W).contiguous()

        return x
```

```
class SpikingSelfAttention(nn.Module):
    def __init__(self, dim, num_heads=8, qkv_bias=False, qk_scale=None,
attn_drop=0., proj_drop=0., sr_ratio=1):
        super().__init__()
        assert dim % num_heads == 0, f"dim {dim} should be divided by num_heads
{num_heads}."
        self.dim = dim
        self.num_heads = num_heads
        self.scale = 0.125
        self.proj_lif = MultiStepLIFNode(tau=2.0, detach_reset=True,
backend='torch')
        self.q_conv = nn.Conv1d(dim, dim, kernel_size=1, stride=1, bias=False)
        self.q_bn = nn.BatchNorm1d(dim)
        self.q_lif = MultiStepLIFNode(tau=2.0, detach_reset=True, backend='torch')

        self.k_conv = nn.Conv1d(dim, dim, kernel_size=1, stride=1, bias=False)
        self.k_bn = nn.BatchNorm1d(dim)
        self.k_lif = MultiStepLIFNode(tau=2.0, detach_reset=True, backend='torch')

        self.v_conv = nn.Conv1d(dim, dim, kernel_size=1, stride=1, bias=False)
        self.v_bn = nn.BatchNorm1d(dim)
        self.v_lif = MultiStepLIFNode(tau=2.0, detach_reset=True, backend='torch')
        self.attn_lif = MultiStepLIFNode(tau=2.0, v_threshold=0.5,
detach_reset=True, backend='torch')

        self.proj_conv = nn.Conv1d(dim, dim, kernel_size=1, stride=1)
        self.proj_bn = nn.BatchNorm1d(dim)


    def forward(self, x):
        T,B,C,H,W = x.shape

        x = self.proj_lif(x)

        x = x.flatten(3)
        T, B, C, N = x.shape
        x_for_qkv = x.flatten(0, 1)
        q_conv_out = self.q_conv(x_for_qkv)
        q_conv_out = self.q_bn(q_conv_out).reshape(T,B,C,N).contiguous()
        q_conv_out = self.q_lif(q_conv_out)
        q = q_conv_out.transpose(-1, -2).reshape(T, B, N, self.num_heads,
C//self.num_heads).permute(0, 1, 3, 2, 4).contiguous()

        k_conv_out = self.k_conv(x_for_qkv)
        k_conv_out = self.k_bn(k_conv_out).reshape(T,B,C,N).contiguous()
        k_conv_out = self.k_lif(k_conv_out)
        k = k_conv_out.transpose(-1, -2).reshape(T, B, N, self.num_heads,
C//self.num_heads).permute(0, 1, 3, 2, 4).contiguous()
```

```
        v_conv_out = self.v_conv(x_for_qkv)
        v_conv_out = self.v_bn(v_conv_out).reshape(T,B,C,N).contiguous()
        v_conv_out = self.v_lif(v_conv_out)
        v = v_conv_out.transpose(-1, -2).reshape(T, B, N, self.num_heads,
C//self.num_heads).permute(0, 1, 3, 2, 4).contiguous()

        x = k.transpose(-2,-1) @ v
        x = (q @ x) * self.scale

        x = x.transpose(3, 4).reshape(T, B, C, N).contiguous()
        x = self.attn_lif(x)
        x = x.flatten(0,1)
        x = self.proj_bn(self.proj_conv(x)).reshape(T,B,C,H,W)

        return x

class SpikingTransformer(nn.Module):
    def __init__(self, dim, num_heads, mlp_ratio=4., qkv_bias=False, qk_scale=None,
drop=0., attn_drop=0.,
                 drop_path=0., norm_layer=nn.LayerNorm, sr_ratio=1):
        super().__init__()
        self.norm1 = norm_layer(dim)
        self.attn = SpikingSelfAttention(dim, num_heads=num_heads,
qkv_bias=qkv_bias, qk_scale=qk_scale,
                                         attn_drop=attn_drop, proj_drop=drop,
sr_ratio=sr_ratio)
        self.drop_path = DropPath(drop_path) if drop_path > 0. else nn.Identity()
        self.norm2 = norm_layer(dim)
        mlp_hidden_dim = int(dim * mlp_ratio)
        self.mlp = MLP(in_features=dim, hidden_features=mlp_hidden_dim, drop=drop)

    def forward(self, x):
        x = x + self.attn(x)
        x = x + self.mlp(x)

        return x

class SpikingTokenizer(nn.Module):
    def __init__(self, img_size_h=128, img_size_w=128, patch_size=4, in_channels=2,
embed_dims=256):
        super().__init__()
        self.image_size = [img_size_h, img_size_w]
        patch_size = to_2tuple(patch_size)
        self.patch_size = patch_size
        self.C = in_channels
        self.H, self.W = self.image_size[0] // patch_size[0], self.image_size[1] //
patch_size[1]
        self.num_patches = self.H * self.W
        self.proj_conv = nn.Conv2d(in_channels, embed_dims//8, kernel_size=3,
stride=1, padding=1, bias=False)
```

```
        self.proj_bn = nn.BatchNorm2d(embed_dims//8)

        self.proj1_lif = MultiStepLIFNode(tau=2.0, detach_reset=True,
backend='torch')
        self.maxpool1 = torch.nn.MaxPool2d(kernel_size=3, stride=2, padding=1,
dilation=1, ceil_mode=False)
        self.proj1_conv = nn.Conv2d(embed_dims//8, embed_dims//4, kernel_size=3,
stride=1, padding=1, bias=False)
        self.proj1_bn = nn.BatchNorm2d(embed_dims//4)

        self.proj2_lif = MultiStepLIFNode(tau=2.0, detach_reset=True,
backend='torch')
        self.maxpool2 = torch.nn.MaxPool2d(kernel_size=3, stride=2, padding=1,
dilation=1, ceil_mode=False)
        self.proj2_conv = nn.Conv2d(embed_dims//4, embed_dims//2, kernel_size=3,
stride=1, padding=1, bias=False)
        self.proj2_bn = nn.BatchNorm2d(embed_dims//2)

        self.proj3_lif = MultiStepLIFNode(tau=2.0, detach_reset=True,
backend='torch')
        self.maxpool3 = torch.nn.MaxPool2d(kernel_size=3, stride=2, padding=1,
dilation=1, ceil_mode=False)
        self.proj3_conv = nn.Conv2d(embed_dims//2, embed_dims, kernel_size=3,
stride=1, padding=1, bias=False)
        self.proj3_bn = nn.BatchNorm2d(embed_dims)

        self.proj4_lif = MultiStepLIFNode(tau=2.0, detach_reset=True,
backend='torch')
        self.maxpool4 = torch.nn.MaxPool2d(kernel_size=3, stride=2, padding=1,
dilation=1, ceil_mode=False)
        self.proj4_conv = nn.Conv2d(embed_dims, embed_dims, kernel_size=3, stride=1,
padding=1, bias=False)
        self.proj4_bn = nn.BatchNorm2d(embed_dims)

    def forward(self, x):
        T, B, C, H, W = x.shape
        x = self.proj_conv(x.flatten(0, 1))
        x = self.proj_bn(x).reshape(T, B, -1, H, W).contiguous()

        x = self.proj1_lif(x).flatten(0,1).contiguous()
        x = self.maxpool1(x)
        x = self.proj1_conv(x)
        x = self.proj1_bn(x).reshape(T, B, -1, H//2, W//2).contiguous()

        x = self.proj2_lif(x).flatten(0, 1).contiguous()
        x = self.maxpool2(x)
        x = self.proj2_conv(x)
        x = self.proj2_bn(x).reshape(T, B, -1, H//4, W//4).contiguous()

        x = self.proj3_lif(x).flatten(0, 1).contiguous()
```

```
        x = self.maxpool3(x)
        x = self.proj3_conv(x)
        x = self.proj3_bn(x).reshape(T, B, -1, H//8, W//8).contiguous()

        x = self.proj4_lif(x).flatten(0, 1).contiguous()
        x = self.maxpool4(x)
        x = self.proj4_conv(x)
        x = self.proj4_bn(x).reshape(T, B, -1, H//16, W//16).contiguous()

        H, W = H // self.patch_size[0], W // self.patch_size[1]
        return x, (H, W)

class vit_snn(nn.Module):
    def __init__(self,
                 img_size_h=128, img_size_w=128, patch_size=16, in_channels=2,
num_classes=11,
                 embed_dims=[64, 128, 256], num_heads=[1, 2, 4], mlp_ratios=[4, 4,
4], qkv_bias=False, qk_scale=None,
                 drop_rate=0., attn_drop_rate=0., drop_path_rate=0.,
norm_layer=nn.LayerNorm,
                 depths=[6, 8, 6], sr_ratios=[8, 4, 2], T = 4, pretrained_cfg= None,
                 **kwargs # Add **kwargs to accept any extra arguments
                 ):
        super().__init__()
        self.num_classes = num_classes
        self.depths = depths
        self.T = T

        #dpr = [x.item() for x in torch.linspace(0, drop_path_rate, depths)]  #
stochastic depth decay rule
        total_depth = sum(depths) if isinstance(depths, list) else depths
        dpr = [x.item() for x in torch.linspace(0, drop_path_rate, total_depth)]  #
stochastic depth decay rule

        patch_embed = SpikingTokenizer(img_size_h=img_size_h,
                                img_size_w=img_size_w,
                                patch_size=patch_size,
                                in_channels=in_channels,
                                embed_dims=embed_dims)

        block = nn.ModuleList([SpikingTransformer(
            dim=embed_dims, num_heads=num_heads, mlp_ratio=mlp_ratios,
qkv_bias=qkv_bias,
            qk_scale=qk_scale, drop=drop_rate, attn_drop=attn_drop_rate,
drop_path=dpr[j],
            norm_layer=norm_layer, sr_ratio=sr_ratios)
            for j in range(depths)])

        setattr(self, f"patch_embed", patch_embed)
        setattr(self, f"block", block)
```

```
        # classification head
        self.head = nn.Linear(embed_dims, num_classes) if num_classes > 0 else
nn.Identity()
        self.apply(self._init_weights)

    @torch.jit.ignore
    def _get_pos_embed(self, pos_embed, patch_embed, H, W):
        if H * W == self.patch_embed1.num_patches:
            return pos_embed
        else:
            return F.interpolate(
                pos_embed.reshape(1, patch_embed.H, patch_embed.W, -1).permute(0, 3,
1, 2),
                size=(H, W), mode="bilinear").reshape(1, -1, H * W).permute(0, 2, 1)

    def _init_weights(self, m):
        if isinstance(m, nn.Linear):
            trunc_normal_(m.weight, std=.02)
            if isinstance(m, nn.Linear) and m.bias is not None:
                nn.init.constant_(m.bias, 0)
        elif isinstance(m, nn.LayerNorm):
            nn.init.constant_(m.bias, 0)
            nn.init.constant_(m.weight, 1.0)

    def forward_features(self, x):

        block = getattr(self, f"block")
        patch_embed = getattr(self, f"patch_embed")

        x, (H, W) = patch_embed(x)
        attn = None
        for blk in block:
            x = blk(x)
        return x.flatten(3).mean(3)

    def forward(self, x):
        x = (x.unsqueeze(0)).repeat(self.T, 1, 1, 1, 1)
        x = self.forward_features(x)
        x = self.head(x.mean(0))
        return x


@register_model
def Spikingformer(pretrained=False, **kwargs):
    # Remove 'pretrained_cfg_overlay' if it's passed, as vit_snn doesn't accept it
    kwargs.pop('pretrained_cfg_overlay', None)
    model = vit_snn(
        **kwargs
    )
```

```
    model.default_cfg = _cfg()
    return model

# register_model
# def Spikingformer(pretrained=False, **kwargs):
#     # Cấu hình MỚI cho 8-768
#     # Lưu ý: Tên biến truyền vào (embed_dims, num_heads, depths...) phải khớp
#     # với hàm __init__ của class vit_snn trong file model.py của bạn.
#     model_kwargs = dict(
#         patch_size=16,
#         embed_dims=768,  # THAY ĐỔI: 512 -> 768
#         num_heads=12,  # THAY ĐỔI: 8 -> 12 (vì 768/12 = 64)
#         mlp_ratios=4,    # MLP hidden sẽ tự động là 768 * 4 = 3072
#         depths=8,
#         **kwargs
#     )
#     model = vit_snn(**model_kwargs)
#     model.default_cfg = _cfg()
#     return model

from timm.models import create_model

if __name__ == '__main__':
    # Check for CUDA availability and set device accordingly
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Using device: {device}")

    x = torch.randn(2, 3, 224, 224).to(device) # Move tensor to the selected device
    model = create_model(
        'Spikingformer',
        img_size_h=224, img_size_w=224,
        patch_size=16, embed_dims=768, num_heads=12, mlp_ratios=4,
        in_channels=3, num_classes=1000, qkv_bias=False,
        norm_layer=partial(nn.LayerNorm, eps=1e-6), depths=8, sr_ratios=1,
        T = 4
    ).to(device) # Move model to the selected device

    model.eval()
    y = model(x)
    print(y.shape)
    print('Test Good!')
```